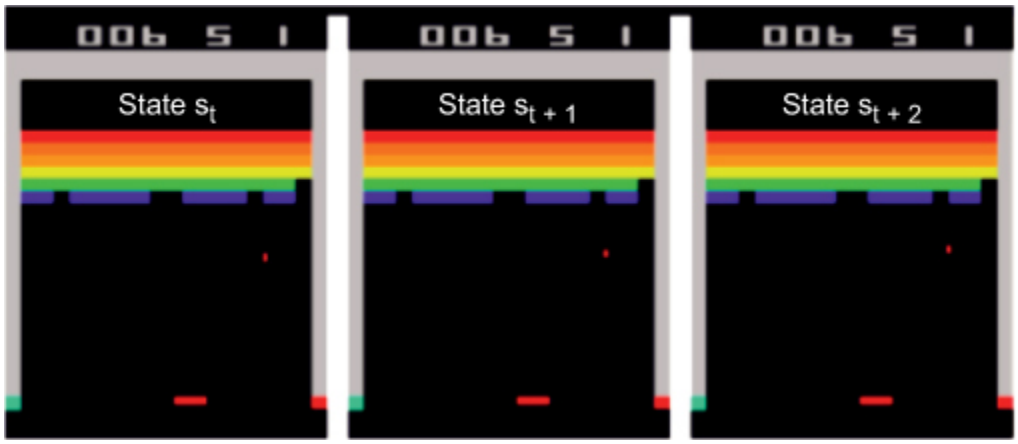


Storing every possible state isn't possible, but we could try to limit the possibilities. In the game Breakout, you control a paddle at the bottom of the screen that can move right or left; the objective of the game is to deflect the ball and break as many blocks at the top of the screen. In that case, we could define constraints—only look at the states when the ball is returning to the paddle, since our actions are not important while we are waiting for the ball at the top of the screen. Or we could provide our own features—instead of providing the raw image, just provide the position of the ball, paddle, and the remaining blocks. However, these methods require the programmer to understand the underlying strategies of the game, and they would not generalize to other environments.

That's where deep learning comes in. A deep learning algorithm can learn to abstract away the details of specific arrangements of pixels and can learn the important features of a state. Since a deep learning algorithm has a finite number of parameters, we can use it to compress any possible state into something we can efficiently process, and then use that new representation to make our decisions. As a result of using neural networks, the Atari DQN only had 1792 parameters (convolutional neural network with 16 8 × 8 filters, 32 4 × 4 filters, and a 256-node fully connected hidden layer) as opposed to the 256^{28228} key/value pairs that would be needed to store the entire state space.

In the case of the Breakout game, a deep neural network might learn on its own to recognize the same high-level features a programmer would have to hand-engineer in a lookup table approach. That is, it might learn how to